



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

*I **Monika.R (192324281)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Annexure A

SCENARIO-BASED TASK

QUESTIONS:

1. Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
2. Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $\text{id} + \text{id} - \text{id}$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.
3. Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + * b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.
4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
5. Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid \text{id}$. While parsing the input $\text{id} + \text{id} * \text{id}$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

CSA1402 - COMPILER DESIGN

R. MONIKA

08/08/2026

192324281

CO2 - AT3 - SCENARIO BASED TASK

Q1. **Scenario:** Parsing Table conflict in LL(1)

Analysis:-

An LL(1) parser predicts the correct production by using the FIRST and FOLLOW sets while constructing the parsing table.

FIRST(A): contains terminals that can appear first in string derived from non-terminal A.

FOLLOW(A): contains terminals that can appear immediately after A.

During parsing table construction:

- If $A \rightarrow \alpha$, place this production in $M[A, a]$, for every $a \in \text{FIRST}(\alpha)$
- If $\epsilon \in \text{FIRST}(\alpha)$, place $A \rightarrow \alpha$ in $M[A, b]$ for every $b \in \text{FOLLOW}(A)$.

A conflict occurs when more than one production is entered into the same table cell, meaning the parser cannot uniquely decide which production to use.

Reason for conflict:

The conflict occurs because the grammar is not LL(1) due to:

- * Common prefixes (Left Factoring required)
- * Left Recursion
- * Overlapping FIRST sets
- * FIRST / FOLLOW conflicts when ϵ -productions exist

Example:-

$$\left. \begin{array}{l} A \rightarrow aB \\ A \rightarrow ac \end{array} \right\} \begin{array}{l} \text{Both production begins with } a, \\ \text{producing multiple entities.} \end{array}$$

Effect on Deterministic Parsing:-

- Parser cannot uniquely choose a production.
- Predictive parsing fails.
- Backtracking becomes necessary.
- Parsing becomes non-deterministic.

Decision:-

Modify the grammar by:

- > Eliminating Left Recursion
- > Applying Left Factoring.
- > Ensuring FIRST and FOLLOW sets do not overlap.

Conclusion:-

An LL(1) grammar must produce a parsing table without conflicts. Removing left recursion and applying left factoring with exactly one production per table entry.

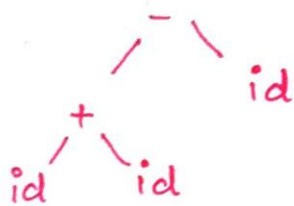
Q2. Scenario: Incorrect Parse Tree Generation

Analysis:- The syntax analyzer constructs a parse tree according to the grammar rules.

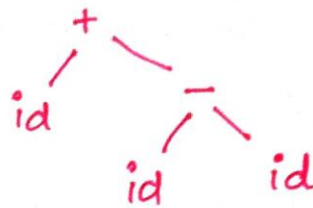
For the expression: $id + id - id$

Possible parse tree

$(id + id) - id$



$id + (id - id)$



Different groupings produce different evaluation results.

Issue:-

- The grammar does not enforce left associativity.
- subtraction and addition are left-associative operators evaluation should occur from left to right.
- Incorrect grouping leads to an incorrect parse tree and wrong computation.

Role of Grammar: Grammar determines the structure of the parse tree.

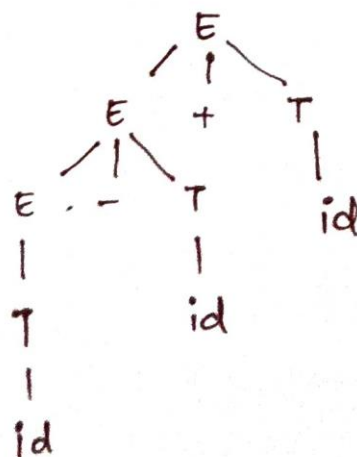
Example:-

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow id$



This grammar naturally generates left-associative parse tree.

Q3. Scenario: Invalid Expression Handling

Analysis:-

Input received: $a + * b$

Token: $id + * id$

Each token is valid individually. However, according to expression grammar, after '+' an operand (id) is expected. Instead, the parser encounters '*'. Hence no valid production can be applied.

Key issue:- The problem is incorrect token arrangement, not invalid tokens.

The grammar expects	"Operand operator operand"
but receives	"operand operator operator operand"

Predictive Parsing

Predictive Parser:

- * Looks up the parsing table
- * Finds no valid production
- * Reports a syntax error immediately.

Shift-Reduce Parsing

Shift-Reduce Parser:

- * Attempts Shifting and Reducing.
- * Eventually reaches a state where no valid action exists.

Decision:-

- Phrase-Level Recovery
- Error production
- Synchronizing Tokens.

These methods allow parsing to continue after detecting an error.

Q4. Scenario: Ambiguity in conditional Statement

Grammar:

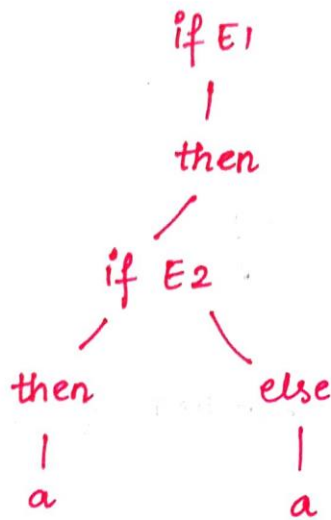
$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

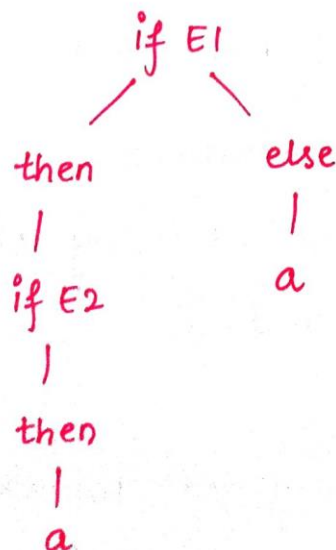
$S \rightarrow a$

Analysis: The parser encounters ambiguity when parsing nested if-else statements.

Example 1.



The parser cannot determine whether else belongs to the first or second if



Key Issue:- This ambiguity is called the Dangling Else Problem

The parser has two valid choices:

- * Associate else with the outer if
- * Associate else with the nearest if.

Hence parsing becomes ambiguous.

Impact:-

- Parsing table conflicts
- shift-reduce conflicts
- Multiple parse tree
- Non-deterministic parsing.

Decision:-

use the Matched - Unmatched Grammar

Example:-

- > Matched $\rightarrow$ if E then matched else Matched / a
- > unmatched $\rightarrow$ if E then S / if E then matched else unmatched
- > S $\rightarrow$ Matched | unmatched.

Alternatively, LR Parsers resolve this by always attaching else to the nearest unmatched if.

Conclusion:-

using matched / unmatched grammar or LR parsing removes ambiguity and correctly associated each else with the nearest unmatched if.

Q5. Scenario: Operator Precedence Conflict

Grammar:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

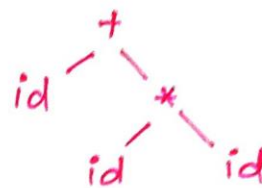
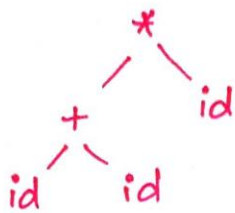
Input: $id + id * id$

Analysis: The grammar is ambiguous because it does not specify operator precedence.

The parser may generate either:

$(id + id) * id$

$id + (id * id)$



Issue:-

- Multiplication ($*$) should have higher precedence than addition ($+$).

- The grammar fails to represent this rule.

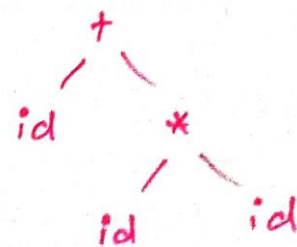
Precedence and Associativity:

correct evaluation: $id + (id * id)$

since $* > +$

Addition and multiplication

are both left associative.



Grammar Modification

Rewrite grammar as :

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow id$$

This grammar naturally enforces:

- Multiplication before addition.
- Left associativity.

Alternatively, parser generators (YACC/Bison) can use precedence and associativity declarations.

Decision :

Modify the grammar to separate precedence levels or define precedence rules in the parser generator.

Conclusion :

using precedence-based grammar eliminates ambiguity, produces the correct parse tree and ensures expressions are evaluated according to standard arithmetic rules.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand